DLA Final Exam Questions — Answer Sheet

> **Comprehensive answers extracted from CT-1 Question Bank**
>
> This document contains detailed answers to the 5 final exam questions.

Question 1: Explain the architecture of Artificial Neural Networks (ANN). Describe the role of activation functions and compare Machine Learning vs Deep Learning with suitable examples.

Answer:

Part A: Architecture of Artificial Neural Networks

Definition

Deep Learning is a subfield of Machine Learning that uses **artificial neural networks with many layers** to learn hierarchical representations from raw data automatically. Deep Learning has revolutionised fields like computer vision, natural language processing, and speech recognition by achieving human-level or superhuman performance on tasks that were previously intractable.

Why is it Called "Deep"?

The term "deep" refers to the **number of hidden layers** in the neural network — its depth. A traditional "shallow" neural network might have just 1 hidden layer. A **deep** neural network has many hidden layers — commonly 10, 50, or even hundreds.

The depth is not just about quantity; it fundamentally changes what the network can learn. Each layer transforms the data into a **more abstract representation**. The network learns a **hierarchy of features** — simple features in early layers, complex concepts in later layers.

Hierarchical Feature Learning (example: face recognition)

```
Layer              What it Detects
──────────────────────────────────────────────────────
Input              Raw pixel values (brightness numbers)
Hidden Layer 1     Simple edges — horizontal, vertical, diagonal lines
Hidden Layer 2     Corners, curves, basic textures
Hidden Layer 3     Eyes, nose, mouth shapes, hair patterns
Hidden Layer 4     Full face regions — forehead, jaw, cheekbones
Output             Identity: "This is Person A"
```

No human programmed these intermediate representations — the network discovered them entirely on its own from millions of labelled examples.

Architecture of a Deep Neural Network

A deep neural network consists of three types of layers:

1. Input Layer Receives the raw data. The number of neurons equals the dimensionality of the input. For a 28×28 grayscale image, the input layer has 784 neurons (one per pixel). For a sentence, it might be token embeddings.

2. Hidden Layers (the "deep" part) Multiple layers of neurons, each applying a weighted transformation followed by a non-linear activation function. Each layer receives the output of the previous layer and produces a new, more abstract representation. The network learns what representations to form — we do not specify them manually.

3. Output Layer Produces the final prediction. For classification, it has one neuron per class; for regression, one neuron for the continuous value. Activation functions here depend on the task (Softmax for multi-class, Sigmoid for binary, linear for regression).

Common Deep Architectures

Architecture	Typical Depth	Primary Use Case
DNN (Dense Deep NN)	3–20 layers	Tabular/structured data
CNN (Convolutional NN)	10–150+ layers	Images, video
RNN / LSTM	Variable	Sequences, time series, speech
Transformer	12–96 layers	NLP, vision (BERT, GPT, ViT)

Why Depth Matters

- The **Universal Approximation Theorem** states that a sufficiently deep (or wide) network can approximate any continuous function to arbitrary precision.
- Depth provides **exponential expressiveness** — a deep network can represent functions that would require exponentially more neurons in a shallow network.
- Depth enables **transfer learning** — lower layers learn general features (edges, textures) that are reusable across tasks; only upper layers need retraining for new tasks.

Part B: Role of Activation Functions

Why Activation Functions are Essential

An activation function is applied to the output of each neuron before passing it to the next layer. The critical reason they exist is to introduce **non-linearity** into the network.

Without activation functions, no matter how many layers a neural network has, the entire network would collapse into a single linear operation — equivalent to just one layer. It could only model straight-line (linear)

relationships between inputs and outputs. Real-world data (images, speech, language) has deeply non-linear structure, so a purely linear network would be fundamentally incapable of learning it.

Activation functions also mimic the **biological firing behaviour** of neurons: a biological neuron only "fires" (sends a signal) if the incoming stimulus crosses a threshold. Similarly, an artificial activation function decides how strongly a neuron responds to its input.

Comparison of Three Common Activation Functions

Property	Sigmoid	Tanh	ReLU
Formula	$\sigma(x) = \dfrac{1}{1+e^{-x}}$	$\tanh(x) = \dfrac{e^x - e^{-x}}{e^x + e^{-x}}$	$f(x) = \max(0, x)$
Output Range	$(0, 1)$	$(-1, 1)$	$[0, \infty)$
Zero-centered output	No — always positive	Yes — centred at 0	No
Vanishing Gradient	Yes — severe at extremes	Yes — at $\pm$ large values	No (for positive inputs)
Dead Neurons	No	No	Yes — can occur for $x \leq 0$
Typical Use	Binary output layer	Hidden layers in RNNs	Hidden layers in CNNs, DNNs

---

1. Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

The sigmoid function produces an S-shaped curve that squashes any real number into the range $(0, 1)$. This makes it perfect for modelling **probabilities** — the output can be interpreted as "the probability of class 1."

Problem — Vanishing Gradient: When x is very large (positive or negative), the sigmoid curve becomes nearly flat. Its derivative at these points is nearly zero. During backpropagation, multiplying near-zero derivatives across many layers causes gradients to shrink exponentially, so early layers learn almost nothing. This is the **vanishing gradient problem**.

---

2. Tanh (Hyperbolic Tangent)

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Tanh is essentially a rescaled version of sigmoid, with output ranging from $(-1, 1)$. The key advantage over sigmoid is that its output is **zero-centred** — meaning roughly half the outputs are positive and half negative. This leads to more stable gradient updates because weight updates don't all push in the same direction.

Problem: Like sigmoid, tanh also saturates at large values, causing vanishing gradients in very deep networks.

---

3. ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

ReLU is the most widely used activation function in modern deep learning. It is elegantly simple: output the input if it is positive, otherwise output zero. For positive inputs, the derivative is exactly 1 — **gradients flow through without shrinking**, solving the vanishing gradient problem that plagued sigmoid and tanh.

ReLU is also computationally very cheap — just a comparison with zero — which matters when training networks with millions of neurons.

Problem — Dead ReLU: If a neuron receives a large negative weight update, its input will always be negative, so it always outputs zero and receives zero gradient. The neuron is "dead" and never recovers during training. This is solved by **Leaky ReLU**, which allows a small negative slope (e.g., $0.01 \times x$) instead of clamping to zero.

Part C: Machine Learning vs Deep Learning Comparison

Definitions

Machine Learning (ML) is a branch of artificial intelligence where algorithms learn patterns from data to make predictions or decisions. The critical characteristic of traditional ML is that it requires **human experts to manually design features** — meaningful representations of the raw data that the algorithm can work with. For example, to classify images of cats and dogs using a traditional ML algorithm, a human would first need to extract features like edge histograms, colour distributions, and texture patterns from the images.

Deep Learning (DL) is a specialised subset of ML that uses **artificial neural networks with many layers**. The defining advantage of DL is that it **automatically learns features directly from raw data** — no manual feature engineering is needed. The network itself discovers which features are important, at multiple levels of abstraction. Given the same raw pixels, a deep learning model will learn to detect edges, then shapes, then object parts, then full objects — entirely on its own.

Comparison Table

Aspect	Machine Learning	Deep Learning
Feature Engineering	Manual — requires domain expertise	Automatic — learned by the network
Data Requirement	Works well with hundreds to thousands of samples	Needs large datasets (tens of thousands to millions)
Computational Cost	Low to moderate — runs on CPU	High — requires GPUs or TPUs

Aspect	Machine Learning	Deep Learning
Interpretability	Models like Decision Trees are interpretable	Neural networks are largely "black boxes"
Performance on Images/Audio	Limited — depends on quality of manual features	Excellent — state of the art
Performance on Tabular/Structured Data	Often better — less data, faster, interpretable	Not always superior for small structured datasets
Training Time	Fast	Slow
Example Algorithms	SVM, Random Forest, Decision Tree, KNN	CNN, RNN, LSTM, Transformer

Suitable Examples

Task	ML Approach	DL Approach
Image Classification	SVM trained on manually extracted HOG (Histogram of Gradients) features	CNN trained directly on raw pixels — automatically learns filters
Sentiment Analysis	TF-IDF bag-of-words + Logistic Regression	LSTM or Transformer (BERT) trained on raw token sequences
Fraud Detection	Random Forest on hand-crafted transaction features (amount, time, location)	Deep neural network that discovers patterns in raw transaction sequences

Key Insight: The Data Size Factor

The relationship between data size and performance is the defining factor in choosing between ML and DL. With small datasets, traditional ML methods are often better because they generalise well with less data and are faster to train. As dataset size grows into the millions, deep learning's ability to automatically extract rich features causes it to significantly outperform traditional ML.

```
Performance
    |          Deep Learning ↗
    |        /
    |      /
    |    / Machine Learning (plateaus)
    |__/_____________________________
                  Data Size →
```

Question 2: Explain Adagrad and RMSProp optimizers. Compare their learning rate adaptation mechanisms and limitations.

Answer:

Part A: Adagrad Optimizer

What is Adagrad?

Adagrad (Adaptive Gradient Algorithm) is an optimization algorithm that solves a key problem with standard gradient descent: it uses the **same learning rate for every parameter**, regardless of how often or how strongly that parameter has been updated.

Adagrad introduces **per-parameter learning rates** that automatically adapt based on the history of gradients seen for each parameter. The core insight is simple:

- A parameter that receives **large or frequent gradients** is already being trained a lot — slow it down by reducing its learning rate.
- A parameter that receives **small or rare gradients** has barely been trained — speed it up by keeping its learning rate large.

This makes Adagrad particularly powerful for **sparse data problems**, such as word embeddings in NLP, where most words appear rarely but a few words appear very often. Without Adagrad, the rare words would be undertrained.

Core Idea: Accumulating History

Adagrad tracks the **sum of squared gradients** for each parameter over all past training steps. The larger this accumulated sum, the smaller the effective learning rate becomes. Think of it as a "budget" — the more you've already updated a parameter, the less aggressively you update it going forward.

Mathematical Formulation

Step 1 — Accumulate squared gradients:

$$G_t = G_{t-1} + \left(\frac{\partial L}{\partial \theta_t}\right)^2$$

What this means: At each time step t, we add the square of the current gradient to a running total G_t. This accumulates the entire training history of how large the gradients were for this parameter.

Symbol	Meaning
G_t	Running sum of squared gradients from step 1 to step t — grows over time
G_{t-1}	The accumulated sum from the previous step
$\frac{\partial L}{\partial \theta_t}$	The gradient of the loss with respect to parameter θ at step t — tells us the direction and magnitude of the error signal
Initialized	$G_0 = 0$ for all parameters at the start

Step 2 — Parameter update:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \varepsilon}} \cdot \frac{\partial L}{\partial \theta_t}$$

What this means: Instead of dividing by a fixed learning rate, Adagrad divides by the square root of the accumulated history. Parameters with large history get a smaller effective step; parameters with small history get a larger step.

Symbol	Meaning
θ_t	The current value of the parameter (weight or bias) at step t
η	The global base learning rate (e.g., 0.01) — same for all parameters initially
$\sqrt{G_t + \varepsilon}$	The adaptive denominator — larger when gradients have been historically large, smaller when they've been small
ε	A tiny constant (typically 10^{-8}) added purely to prevent division by zero when $G_t \approx 0$
$\frac{\partial L}{\partial \theta_t}$	The gradient of the loss — tells us which direction to move the parameter

---

Algorithm Steps

1. Initialize all parameters θ randomly and set $G = 0$ for each parameter.
2. For each training iteration t:
   - Compute the forward pass to get predictions.
   - Calculate the loss L.
   - Compute the gradient g_t for each parameter.
   - Accumulate: $G_t = G_{t-1} + g_t^2$
   - Update each parameter: $\theta_{t+1} = \theta_t - \dfrac{\eta}{\sqrt{G_t + \varepsilon}} \cdot g_t$
3. Repeat until training converges.

Advantages and Disadvantages

Advantages	Disadvantages
No manual per-parameter tuning of learning rates	G_t grows **monotonically** — it never decreases, so the learning rate eventually shrinks to near zero, causing training to stall
Works very well for sparse data (NLP, recommendation)	Not suitable for non-convex deep networks in long runs
Simple to implement and understand	This limitation is directly addressed by RMSProp (which uses a decaying average instead of a cumulative sum) and Adam

---

Part B: RMSProp Optimizer

Adagrad's Limitation: The Dying Learning Rate

Before understanding RMSProp, it's important to understand **why Adagrad fails** in practice for deep networks.

Adagrad accumulates the **sum of all squared gradients** from the very beginning of training into a variable G_t. This sum only ever grows — it never decreases. As training continues over thousands of iterations, G_t becomes very large. Since the effective learning rate is $\frac{\eta}{\sqrt{G_t}}$, a very large G_t makes this fraction extremely small. Eventually, the learning rate shrinks so close to zero that **the model essentially stops learning** — even if it hasn't converged yet. This is particularly damaging in deep learning where training runs for many epochs.

RMSProp: The Fix

RMSProp (Root Mean Square Propagation) was proposed by Geoffrey Hinton to fix this exact problem. Instead of accumulating all past squared gradients into a growing sum, RMSProp uses an **Exponential Moving Average (EMA)** — a weighted average that gives **more importance to recent gradients** and gradually forgets old ones.

Think of it like a memory with **fading recollection**: what happened 10,000 steps ago barely affects the current learning rate, but what happened 10 steps ago matters a lot. This prevents the denominator from growing indefinitely.

Mathematical Formulation

Step 1 — Compute the Exponential Moving Average of squared gradients:

$$E[g^2]*t = |rho |cdot E[g^2]*{t-1} + (1 - \rho) \cdot g_t^2$$

What this means:

- At each step, we combine the **old average** (multiplied by ρ, which keeps most of it) with the **new squared gradient** (multiplied by $1 - \rho$, which adds a small new contribution).
- The decay rate ρ controls the "memory window." With $\rho = 0.9$, about 10 recent steps contribute meaningfully; with $\rho = 0.99$, about 100 steps contribute.
- Crucially, this value **stays bounded** — it cannot grow to infinity because old values are continuously discounted.

Symbol	Meaning
$E[g^2]_t$	The moving average of squared gradients at step t — bounded, unlike Adagrad's G_t
ρ	Decay rate, typically 0.9 — how much of the old average to keep
$(1 - \rho)$	How much weight to give the current gradient — typically 0.1
g_t	The gradient of the loss at step t
g_t^2	Squared gradient — always positive, capturing gradient magnitude

---

Step 2 — Parameter update:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \varepsilon}} \cdot g_t$$

What this means: The update rule looks identical to Adagrad's, but the denominator uses the **bounded moving average** instead of the ever-growing cumulative sum. This keeps the effective learning rate stable throughout training.

Symbol	Meaning
η	Global learning rate (e.g., 0.001)
ε	Small constant (~10^{-8}) to prevent division by zero

Part C: Comparison of Adagrad vs RMSProp

Aspect	Adagrad	RMSProp
Gradient history mechanism	Cumulative sum of all squared gradients — monotonically increases	Exponential moving average — stays bounded
Effective learning rate over time	Shrinks toward zero over time	Remains stable throughout training
Suited for	Short training, sparse/convex problems	Long training, non-convex deep networks
Memory of old gradients	Permanent — never forgets	Fades over time (controlled by ρ)
Main limitation	Learning rate decay too aggressive — training stalls	Requires tuning ρ decay parameter
Typical use case	NLP with sparse features, recommendation systems	Modern deep learning (CNNs, RNNs)
Formula difference	$G_t = G_{t-1} + g_t^2$	$E[g^2]*t = \rho \cdot E[g^2]*{t-1} + (1-\rho) \cdot g_t^2$

Key Insight

The fundamental difference is that **Adagrad has perfect memory** (remembers all past gradients forever), while **RMSProp has selective memory** (prioritises recent gradients). This makes RMSProp much better suited for deep neural networks trained over many iterations.

Question 3: Explain the Gradient Descent algorithm and compare Batch, Stochastic, and Mini-Batch Gradient Descent. Discuss when each should be used.

Answer:

Part A: Gradient Descent Algorithm

What is Gradient Descent?

Gradient Descent is the fundamental optimization algorithm used to train neural networks. Its goal is to find the set of model parameters (weights and biases) that minimise a loss function. The loss function measures how wrong the model's predictions are compared to the true labels.

The core idea is beautifully simple: if you're standing on a mountainside in thick fog and want to descend to the valley floor, take repeated small steps in the direction of steepest descent. Eventually, you'll reach a low point.

Mathematically, at each step, we compute the **gradient** of the loss function with respect to each parameter. The gradient is a vector that points in the direction of steepest **increase**. To minimise the loss, we move in the opposite direction — downhill.

The Update Rule

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla_\theta L$$

Symbol	Meaning
θ_t	Current value of model parameters (weights/biases) at step t
θ_{t+1}	Updated parameters after taking one step
η	Learning rate — controls the step size (e.g., 0.01)
$\nabla_\theta L$	Gradient of the loss function with respect to θ — the direction of steepest ascent

The negative sign means we move **opposite** to the gradient direction — downhill toward lower loss.

Part B: Three Variants of Gradient Descent

All three methods use the same update rule. The fundamental difference between them is **how much of the training data is used to compute the gradient $\nabla_\theta L$ at each update step**. This choice creates important trade-offs between computational efficiency, memory usage, and training stability.

---

1. Batch Gradient Descent (BGD)

In Batch Gradient Descent, **every single training sample** is used to compute the gradient before making one parameter update. The gradient is the exact gradient of the loss over the entire dataset.

Intuition: Before taking any step, you survey the entire landscape from a helicopter to determine the perfect direction downhill. Every step is optimal but takes enormous time to compute.

Mathematical formulation:

$$\nabla_\theta L = \frac{1}{N} \sum_{i=1}^{N} \nabla_\theta L_i$$

Where N is the total number of training samples, and L_i is the loss for sample i.

Characteristics:

- Produces the most accurate gradient estimate — the direction of update is always correct.
- But: for a dataset with millions of examples, computing the gradient over all samples before taking one step is prohibitively slow.
- Requires loading the entire dataset into memory simultaneously.
- Since the gradient is exact (no noise), convergence is smooth and monotonic.
- One complete pass through the dataset (one epoch) = one parameter update.

When to use:

- **Small datasets** (hundreds or a few thousand samples)
- **Convex optimization problems** where the exact gradient is valuable
- When you have sufficient memory to load the entire dataset

---

2. Stochastic Gradient Descent (SGD)

In Stochastic Gradient Descent, a **single randomly chosen training sample** is used to compute the gradient and immediately update the parameters.

Intuition: Instead of surveying the whole landscape, you look at the slope directly under your feet right now and take a step. You update instantly after each sample, so updates are extremely fast — but based on a single, noisy measurement.

Mathematical formulation:

$$\nabla_\theta L \approx \nabla_\theta L_i$$

Where i is a randomly selected training sample.

Characteristics:

- Very fast updates — the model parameters change after every single example.
- The gradient from one sample is a noisy estimate of the true gradient. This noisiness causes the training path to zigzag erratically rather than descend smoothly.
- This noise can actually be **beneficial** — it helps the model escape sharp local minima and find flatter, more generalisable minima (an effect called "implicit regularization").
- Memory efficient — only one sample in memory at a time.
- Used in **online learning**, where data arrives as a stream.

When to use:

- **Online learning** scenarios (data arrives continuously)
- **Very large datasets** where even one epoch of BGD is too expensive
- When you want the noise to help escape poor local minima
- **Sparse updates** (only parameters relevant to the current sample are updated)

---

3. Mini-Batch Gradient Descent

In Mini-Batch Gradient Descent, a **small random subset** (a "mini-batch", typically 32–256 samples) is used to compute the gradient at each step. This is the **standard method used in all modern deep learning**.

Intuition: Instead of the whole landscape or just one footstep, you sample a small local region — enough to get a reasonable direction without needing the full dataset.

Mathematical formulation:

$$\nabla_\theta L \approx \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta L_i$$

Where B is the batch size (e.g., 32, 64, 128, 256).

Characteristics:

- The gradient is a good (though not perfect) approximation of the true gradient. Less noisy than SGD, less expensive than Batch GD.
- Allows **GPU parallelism** — modern GPUs are designed to process batches of data simultaneously, making mini-batch computation very efficient.
- Offers the noise benefits of SGD (helping escape local minima) while being more stable and directional.
- The batch size is a hyperparameter: smaller batches → noisier but faster; larger batches → more stable but slower.

When to use:

- **Modern deep learning** (CNNs, RNNs, Transformers) — this is the default
- Datasets of any size (small to extremely large)
- When you have **GPU hardware** that can exploit parallelism
- When you want a good balance between speed, stability, and generalisation

---

Part C: Comparison Table

Aspect	Batch GD	Stochastic GD	Mini-Batch GD
Data per update	Full dataset (N samples)	1 sample	Batch of B samples (32–256)
Gradient quality	Exact (no noise)	Very noisy	Good approximation
Updates per epoch	1	N	N / B
Convergence path	Smooth, monotonic	Erratic, zigzag	Mostly smooth with some noise
Memory requirement	High (entire dataset)	Very low (one sample)	Moderate (one batch)
GPU utilisation	Good	Poor (too small)	Excellent (optimal batch size)

Aspect	Batch GD	Stochastic GD	Mini-Batch GD
Speed per update	Slow	Fast	Fast
Escape local minima	Poor	Good (noise helps)	Good
Typical use	Small datasets, convex problems	Online learning, streaming data	Modern deep learning (default)

Part D: When to Use Which Variant

Use Batch Gradient Descent when:

- You have a **small dataset** (hundreds to a few thousand samples)
- You have enough **memory** to load the entire dataset
- The problem is **convex** (one global minimum) and the exact gradient is valuable
- You need **deterministic, reproducible** training runs

Use Stochastic Gradient Descent when:

- You're doing **online learning** (data arrives as a stream)
- You have **extremely large datasets** where even one pass through the data takes too long
- You want the **noise to help exploration** and escape sharp minima
- You need **sparse updates** (e.g., only update embeddings for words that appeared in the current sentence)

Use Mini-Batch Gradient Descent when:

- You're training **modern deep neural networks** (CNN, RNN, Transformer)
- You have **GPU hardware** that can exploit parallelism
- You're working with **datasets of any size** (small to extremely large)
- You want the **best practical balance** between speed, stability, memory efficiency, and generalisation

In practice: Mini-Batch Gradient Descent with batch sizes between 32 and 256 is used in approximately 99% of modern deep learning applications. It has become the de facto standard because it perfectly exploits GPU architecture while maintaining good convergence properties.

Question 4: Explain TensorFlow data structures in detail. Discuss tensors, shape, type, variables, constants, and computational graph with suitable examples.

Answer:

Part A: Introduction to TensorFlow

What is TensorFlow?

TensorFlow is an open-source deep learning framework developed by Google. Every computation in TensorFlow — from storing data to performing matrix multiplications to updating weights — is built around a single fundamental data structure: the **tensor**.

The name reflects its core architecture: **tensors** (multi-dimensional arrays) **flow** through a computational graph of mathematical operations.

Part B: What is a Tensor?

A **tensor** is a generalisation of numbers, vectors, and matrices to any number of dimensions. The "rank" of a tensor describes how many dimensions it has. You can think of tensors as containers that hold numbers arranged in a particular shape.

Types of Tensors by Rank

Rank	Name	Shape	Real-World Example
0	**Scalar**	No dimensions	A single number — e.g., the loss value `0.35`, a temperature reading, or a single accuracy score
1	**Vector**	`(n,)` — a list	A row of features — e.g., `[age, height, weight]` for one patient; or one word embedding
2	**Matrix**	`(m, n)` — rows × columns	A table of data — e.g., a weight matrix connecting 128 neurons to 64 neurons; a batch of 32 samples each with 10 features
3+	**n-D Tensor**	`(batch, h, w, c)`	An image batch — e.g., 32 colour images of size 224×224 with 3 channels (RGB) stored as a 4D tensor

Key insight: The rank tells you how many indices you need to point to a single number inside the tensor. A scalar needs zero indices; a vector needs one; a matrix needs two; an image batch needs four.

Part C: TensorFlow Tensor Types (by mutability)

1. tf.constant — Immutable Tensor

A `tf.constant` is a tensor whose value is **fixed** once created and can never be changed. It is used for data that should not be modified during training, such as input features, labels, or pre-defined lookup tables.

Example: Storing the raw training images as a constant ensures they are not accidentally modified.

Code example:

```
import tensorflow as tf

# Create a constant scalar
```

```
scalar = tf.constant(3.14)
print(scalar)  # tf.Tensor(3.14, shape=(), dtype=float32)

# Create a constant vector
vector = tf.constant([1, 2, 3, 4, 5])
print(vector)  # tf.Tensor([1 2 3 4 5], shape=(5,), dtype=int32)

# Create a constant matrix
matrix = tf.constant([[1, 2], [3, 4], [5, 6]])
print(matrix)  # shape=(3, 2)

# Create a 3D tensor (batch of 2 grayscale 4x4 images)
tensor_3d = tf.constant([
    [[0.1, 0.2, 0.3, 0.4],
     [0.5, 0.6, 0.7, 0.8],
     [0.1, 0.2, 0.3, 0.4],
     [0.5, 0.6, 0.7, 0.8]],
    [[0.9, 0.8, 0.7, 0.6],
     [0.5, 0.4, 0.3, 0.2],
     [0.9, 0.8, 0.7, 0.6],
     [0.5, 0.4, 0.3, 0.2]]
])
print(tensor_3d.shape)  # (2, 4, 4)
```

---

2. tf.Variable — Mutable Tensor

A `tf.Variable` is a tensor whose value **can be updated** during training. It is the primary way TensorFlow stores **trainable parameters** — weights and biases. After each training step, gradient descent updates the Variable's value. The Variable "remembers" its value across multiple operations.

Code example:

```
# Initialize a variable (typically used for weights)
weight = tf.Variable([[1.0, 2.0], [3.0, 4.0]], dtype=tf.float32)
print(weight)  # <tf.Variable 'Variable:0' shape=(2, 2) dtype=float32>

# Modify the variable
weight.assign([[5.0, 6.0], [7.0, 8.0]])
print(weight.numpy())  # [[5. 6.] [7. 8.]]

# Update using addition (simulating a gradient descent step)
weight.assign_add([[-0.1, -0.1], [-0.1, -0.1]])
print(weight.numpy())  # [[4.9 5.9] [6.9 7.9]]
```

Key difference from constants: Variables are designed to be modified in-place. Constants cannot be changed after creation.

---

3. tf.SparseTensor — Sparse Tensor

A `tf.SparseTensor` is designed for tensors where **most values are zero**. Instead of storing every zero explicitly (which wastes memory), it stores only the non-zero values along with their positions (indices). This is very useful in NLP — for example, a one-hot encoded vocabulary of 50,000 words is mostly zeros.

Example: Representing the sentence "I love NLP" in a vocabulary of 10,000 words:

- Dense representation: 10,000 elements, 9,997 are zeros
- Sparse representation: Store only `[(0, 42), (1, 157), (2, 8934)]` → indices and values

Code example:

```
# Create a sparse tensor
# indices: [[row, col], ...], values: [val, ...], shape: (rows, cols)
sparse = tf.SparseTensor(
    indices=[[0, 1], [1, 3], [2, 0]],  # positions of non-zero values
    values=[10, 20, 30],                # the non-zero values
    dense_shape=[3, 4]                  # overall tensor shape (3 rows, 4
cols)
)
print(sparse)

# Convert to dense to visualize
dense = tf.sparse.to_dense(sparse)
print(dense.numpy())
# [[ 0 10  0  0]
#  [ 0  0  0 20]
#  [30  0  0  0]]
```

---

4. tf.RaggedTensor — Variable-Length Tensor

A `tf.RaggedTensor` handles data where **rows have different lengths**. For example, sentences in a text dataset are of different lengths: one sentence has 8 words, another has 20. A regular tensor would need padding to make all rows equal length. A RaggedTensor stores them as-is, saving memory and preserving the original structure.

Code example:

```
# Sentences with varying lengths (word IDs)
sentences = tf.ragged.constant([
    [42, 157, 8934],           # Sentence 1: 3 words
    [11, 22, 33, 44, 55],      # Sentence 2: 5 words
    [99, 88]                   # Sentence 3: 2 words
])
print(sentences)
# <tf.RaggedTensor [[42, 157, 8934], [11, 22, 33, 44, 55], [99, 88]]>

print(sentences[0])  # tf.Tensor([  42  157 8934], shape=(3,),
dtype=int32)
```

```
print(sentences[1])  # tf.Tensor([11 22 33 44 55], shape=(5,),
dtype=int32)
```

Part D: Key Properties of Every Tensor

1. dtype (Data Type)

Specifies what kind of numbers the tensor holds. Choosing the right dtype affects both memory usage and numerical precision.

Common dtypes:

- `float32` — 32-bit floating point (default for neural network weights)
- `float64` — 64-bit double precision (for scientific computing requiring high accuracy)
- `int32` — 32-bit integers (for labels, indices)
- `bool` — Boolean values (masks, conditions)

Example:

```
tensor_float = tf.constant([1.0, 2.0, 3.0], dtype=tf.float32)
tensor_int = tf.constant([1, 2, 3], dtype=tf.int32)
print(tensor_float.dtype)  # <dtype: 'float32'>
print(tensor_int.dtype)    # <dtype: 'int32'>
```

---

2. shape

Describes the size of each dimension — e.g., `(32, 224, 224, 3)` means 32 images, each 224×224 pixels, with 3 colour channels.

Example:

```
tensor = tf.constant([[1, 2, 3], [4, 5, 6]])
print(tensor.shape)  # (2, 3) — 2 rows, 3 columns
print(tensor.shape[0])  # 2 — number of rows
print(tensor.shape[1])  # 3 — number of columns
```

---

3. rank

The number of dimensions — `len(shape)`.

Example:

```
scalar = tf.constant(5)
vector = tf.constant([1, 2, 3])
matrix = tf.constant([[1, 2], [3, 4]])

print(tf.rank(scalar))  # 0 — scalar
print(tf.rank(vector))  # 1 — vector
print(tf.rank(matrix))  # 2 — matrix
```

Part E: Computational Graph

What is a Computational Graph?

A **computational graph** is a directed acyclic graph (DAG) that represents the mathematical operations (nodes) and the data flow (edges) between them. Each node is an operation (addition, matrix multiplication, etc.), and each edge is a tensor flowing from one operation to another.

In TensorFlow 1.x: The graph had to be explicitly defined, then executed in a separate step (called "session").

In TensorFlow 2.x (current): The framework uses **Eager Execution** by default — operations execute immediately as you write them, like normal Python code. The graph is built implicitly behind the scenes for optimization and deployment.

Example: Simple Computational Graph

```
import tensorflow as tf

# Define operations
a = tf.constant(3.0)
b = tf.constant(4.0)
c = a + b
d = c * 2
e = d ** 2

print(e.numpy())  # 196.0

# The computational graph:
#     a(3) ──┐
#            ├──→ c(7) ──→ d(14) ──→ e(196)
#     b(4) ──┘
```

Use of tf.GradientTape for Automatic Differentiation

tf.GradientTape records operations during the forward pass to enable automatic differentiation (backpropagation).

```
x = tf.Variable(3.0)

with tf.GradientTape() as tape:
    y = x ** 2  # y = 9.0

# Compute gradient dy/dx
grad = tape.gradient(y, x)
print(grad.numpy())  # 6.0 (derivative of x^2 is 2x = 2*3 = 6)
```

Real training loop example:

```
# Model parameters
w = tf.Variable(0.5)
b = tf.Variable(0.0)

# Simple linear model: y = w*x + b
def model(x):
    return w * x + b

# Training data
x_train = tf.constant([1.0, 2.0, 3.0, 4.0])
y_train = tf.constant([2.0, 4.0, 6.0, 8.0])  # y = 2*x

# Training loop
learning_rate = 0.01
for epoch in range(100):
    with tf.GradientTape() as tape:
        y_pred = model(x_train)
        loss = tf.reduce_mean((y_pred - y_train) ** 2)  # MSE

    # Compute gradients
    grads = tape.gradient(loss, [w, b])

    # Update parameters
    w.assign_sub(learning_rate * grads[0])
    b.assign_sub(learning_rate * grads[1])

    if epoch % 20 == 0:
        print(f"Epoch {epoch}, Loss: {loss.numpy():.4f}, w:
{w.numpy():.4f}, b: {b.numpy():.4f}")

# Final parameters should be close to w=2, b=0
print(f"Learned: w={w.numpy():.4f}, b={b.numpy():.4f}")
```

Summary Table

Tensor Type	Mutability	Primary Use Case	Example

Tensor Type	Mutability	Primary Use Case	Example
tf.constant	Immutable	Input data, labels, fixed values	`tf.constant([1, 2, 3])`
tf.Variable	Mutable	Model parameters (weights, biases)	`tf.Variable([[1.0, 2.0]])`
tf.SparseTensor	Immutable	Sparse data (mostly zeros)	One-hot encoded vocabulary
tf.RaggedTensor	Immutable	Variable-length sequences	Sentences with different word counts

Key Properties:

- **dtype**: Data type (float32, int32, bool)
- **shape**: Dimensions (e.g., `(32, 224, 224, 3)`)
- **rank**: Number of dimensions

Computational Graph: Operations flow through a DAG; `tf.GradientTape` enables automatic differentiation for training.

Question 5: Explain the role of loss functions in neural networks. Differentiate between Binary Cross-Entropy and Categorical Cross-Entropy with use cases.

Answer:

Part A: Role of Loss Functions in Neural Networks

What is a Loss Function?

A **loss function** (also called a cost function or objective function) quantifies how wrong a model's predictions are compared to the true labels. It produces a single number that the optimizer tries to minimize. The loss function is the bridge between what the model predicts and what we want it to learn.

Core principle: During training, we compute the loss on the training data, then use backpropagation to calculate gradients, and finally update the model parameters to reduce that loss. The choice of loss function directly determines what the model optimizes for.

Why Loss Functions Matter

1. Defines the Learning Objective The loss function tells the model what "good" and "bad" mean. Different loss functions encode different notions of error:

- **Mean Squared Error (MSE)**: Penalises large errors heavily (quadratic)
- **Mean Absolute Error (MAE)**: Treats all errors proportionally (linear)
- **Cross-Entropy**: Measures information-theoretic surprise; heavily penalises confident wrong predictions

2. Guides the Optimization Process The optimizer (SGD, Adam, RMSProp) follows the gradient of the loss function. The loss landscape — its valleys, ridges, and plateaus — determines how easily the optimizer can find good solutions. A well-chosen loss function creates a smoother landscape with clearer paths to the minimum.

3. Task-Specific Different tasks require different loss functions:

- **Regression** (predicting continuous values): MSE, MAE, Huber Loss
- **Binary classification**: Binary Cross-Entropy
- **Multi-class classification**: Categorical Cross-Entropy
- **Multi-label classification**: Binary Cross-Entropy (applied per label)

Mathematical Foundation: Cross-Entropy

Cross-entropy loss functions are based on **information theory** — specifically, they measure how surprised we would be by the true label given the model's predicted probabilities.

Key intuition: The logarithm in cross-entropy is what makes it penalise confident wrong predictions so heavily.

- $\log(0.01) \approx -4.6$, but $\log(0.99) \approx -0.01$
- Being confidently wrong is penalised nearly **460× more** than being confidently correct

This creates a strong gradient signal that pushes the model to be both accurate and calibrated in its confidence.

Part B: Binary Cross-Entropy (BCE)

When to Use

Used when there are exactly **two classes** (0 or 1). The output neuron uses a **Sigmoid** activation to produce a probability between 0 and 1.

Examples:

- Spam vs not spam
- Disease vs healthy
- Click vs no click
- Fraud vs legitimate transaction

Mathematical Formula

$$L_{BCE} = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right]$$

Symbol-by-Symbol Explanation

Symbol	Meaning

Symbol	Meaning
N	Total number of training samples being averaged over
y_i	The true label for sample i — either 0 (negative class) or 1 (positive class)
$\hat{y}_i$	The model's predicted probability that sample i belongs to class 1 (output of sigmoid)
$\log(\hat{y}_i)$	The term that applies when $y_i = 1$ — penalises the model for low predicted probability when the true label is 1
$\log(1 - \hat{y}_i)$	The term that applies when $y_i = 0$ — penalises the model for high predicted probability when the true label is 0

How the Two Terms Work Together

The formula cleverly switches between two terms depending on the true label:

- When $y_i = 1$: The second term vanishes ($1 - 1 = 0$) and only $\log(\hat{y}_i)$ matters
- When $y_i = 0$: The first term vanishes and only $\log(1 - \hat{y}_i)$ matters

We never compute both terms for the same sample.

Worked Example: Spam Detection

Case 1: Correct prediction

- True label: spam ($y = 1$)
- Prediction: $\hat{y} = 0.95$
- Loss: $-\log(0.95) \approx 0.05$ — correctly confident, low penalty

Case 2: Wrong but uncertain

- True label: spam ($y = 1$)
- Prediction: $\hat{y} = 0.60$
- Loss: $-\log(0.60) \approx 0.51$ — unsure, moderate penalty

Case 3: Confidently wrong

- True label: spam ($y = 1$)
- Prediction: $\hat{y} = 0.05$
- Loss: $-\log(0.05) \approx 3.0$ — confidently wrong, **high penalty**

Case 4: Correct negative

- True label: not spam ($y = 0$)
- Prediction: $\hat{y} = 0.10$
- Loss: $-\log(1 - 0.10) = -\log(0.90) \approx 0.11$ — correctly confident, low penalty

---

Code Example (PyTorch/TensorFlow)

TensorFlow:

```
import tensorflow as tf

# True labels and predictions
y_true = tf.constant([1, 0, 1, 0], dtype=tf.float32)
y_pred = tf.constant([0.9, 0.2, 0.7, 0.3], dtype=tf.float32)

# Binary cross-entropy
bce = tf.keras.losses.BinaryCrossentropy()
loss = bce(y_true, y_pred)
print(f"Loss: {loss.numpy():.4f}")
```

Part C: Categorical Cross-Entropy (CCE)

When to Use

Used when there are **more than two classes** (multi-class classification where each sample belongs to exactly one class). The output layer uses **Softmax** activation, which converts raw scores into a probability distribution over all classes (all probabilities sum to 1).

Labels are typically **one-hot encoded** (e.g., class 3 out of 5 → `[0, 0, 0, 1, 0]`).

Examples:

- Handwritten digit recognition (0-9)
- Object classification (cat, dog, car, etc.)
- Language identification (English, Spanish, French, etc.)
- Sentiment classification (positive, neutral, negative)

Mathematical Formula

$$L_{CCE} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{ic} \log(\hat{y}_{ic})$$

Symbol-by-Symbol Explanation

Symbol	Meaning
C	Total number of classes (e.g., 10 for digit recognition)
y_{ic}	1 if sample i truly belongs to class c, else 0 (one-hot encoding — only one class is 1 per sample)
$\hat{y}_{ic}$	The predicted probability that sample i belongs to class c (output of Softmax)

Simplification

Because y_{ic} is one-hot, for any given sample, only the term corresponding to the true class is non-zero. The loss reduces to:

$$L_{CCE} = -\log(\hat{y}_{i, \text{true class}})$$

This means we only penalise the model for assigning low probability to the correct class.

Worked Example: Digit Recognition

Suppose we're classifying handwritten digits (0-9), and the true digit is **3**.

True label (one-hot): `[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]`

Case 1: Good prediction

```
Predicted probabilities: [0.01, 0.02, 0.05, 0.85, 0.02, 0.01, 0.01, 0.01,
0.01, 0.01]
                                            ↑
                                        class 3
Loss = −log(0.85) ≈ 0.16 — correctly confident, low penalty
```

Case 2: Uncertain prediction

```
Predicted probabilities: [0.05, 0.10, 0.15, 0.40, 0.10, 0.05, 0.05, 0.05,
0.03, 0.02]
                                            ↑
                                        class 3
Loss = −log(0.40) ≈ 0.92 — correct but uncertain, moderate penalty
```

Case 3: Wrong prediction

```
Predicted probabilities: [0.05, 0.10, 0.15, 0.05, 0.10, 0.05, 0.05, 0.40,
0.03, 0.02]
                                            ↑               ↑
                                        class 3     (predicted class 7)
Loss = −log(0.05) ≈ 3.0 — confidently wrong, high penalty
```

Code Example (PyTorch/TensorFlow)

TensorFlow:

```python
import tensorflow as tf

# True labels (class indices: 0-2)
y_true = tf.constant([0, 1, 2, 0], dtype=tf.int32)

# Predicted probabilities (4 samples, 3 classes)
y_pred = tf.constant([
    [0.7, 0.2, 0.1],  # Sample 0: mostly class 0 ✓
    [0.1, 0.8, 0.1],  # Sample 1: mostly class 1 ✓
    [0.2, 0.3, 0.5],  # Sample 2: mostly class 2 ✓
    [0.4, 0.5, 0.1]   # Sample 3: predicted class 1, but true is 0 ✗
], dtype=tf.float32)

# Categorical cross-entropy
cce = tf.keras.losses.SparseCategoricalCrossentropy()
loss = cce(y_true, y_pred)
print(f"Loss: {loss.numpy():.4f}")
```

Note: `SparseCategoricalCrossentropy` accepts integer labels (0, 1, 2). If you have one-hot encoded labels, use `CategoricalCrossentropy`.

Part D: Comparison Table

Aspect	Binary Cross-Entropy	Categorical Cross-Entropy
Number of classes	2 (binary)	$C > 2$ (multi-class)
Output activation	Sigmoid (produces one probability)	Softmax (produces C probabilities summing to 1)
Output shape	Single neuron, scalar probability	C neurons, probability vector
Label format	Single scalar: 0 or 1	One-hot vector of length C (or integer class index)
Formula	$-[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$	$-\sum_{c=1}^{C} y_c \log(\hat{y}_c)$
When predicted probability is 0	Loss → ∞ (heavily penalised)	Loss → ∞ (heavily penalised)
Example use case	Spam detection, fraud detection, click prediction	Digit recognition, object classification, language ID
Multi-label support	Yes (apply per label independently)	No — assumes exactly one true class

---

Part E: Visual Intuition

Loss Landscape for Binary Cross-Entropy

```
Loss
  ∞|      /                    \
   |     /                      \
   |    /                        \
  3|   /                          \
   |  /                            \
  0|_____________________________|___
   0   0.2   0.4   0.6   0.8   1.0  Predicted Probability

   When y=1:
   - Predicting 1.0 → Loss = 0
   - Predicting 0.5 → Loss ≈ 0.69
   - Predicting 0.0 → Loss = ∞
```

The loss grows exponentially as the predicted probability moves away from the true label. This creates strong gradients that guide the optimizer toward correct predictions.

Part F: Key Takeaways

1. **Loss functions define what the model optimizes for** — they convert the abstract goal of "learning" into a concrete mathematical objective.
2. **Binary Cross-Entropy** is for *two-class problems* with sigmoid output, measuring how well the model predicts a single probability.
3. **Categorical Cross-Entropy** is for *multi-class problems* with softmax output, measuring how well the model distributes probability mass across all classes.
4. Both heavily penalise **confident wrong predictions** due to the logarithm, which creates strong gradient signals for learning.
5. The choice of loss function must match the problem structure:
    - Wrong loss → poor convergence or incorrect learning objective
    - Correct loss → smooth optimization and good generalization

---

End of Answer Sheet